Python Programming Interview Questions (2026)

Advanced Questions That Test Real Engineering Thinking

Written by: *Tanu Nanda Prabhu*

Most Python interview guides focus on syntax and beginner tricks. However, modern interviews in **2026** evaluate something deeper, **how well you understand Python's internals, performance trade-offs, and real-world design decisions.**

Below are **advanced Python interview questions with concise explanations and code examples** that reflect the type of discussions happening in serious engineering interviews today.

1. What actually happens when Python executes a function call?

Python does more than simply "run a function". Internally it creates a **new stack frame**, manages local variables, and pushes execution to the **Python Virtual Machine (PVM)**.

Key Concepts

- Python compiles `.py` code into **bytecode**
- Bytecode runs on the **Python Virtual Machine**
- Each function call creates a **frame object**

Example

```
def add(a, b):
    return a + b

add(3, 4)
```

Internally Python performs steps similar to:

1. Create stack frame
2. Bind `a=3`, `b=4`
3. Execute bytecode
4. Return result
5. Destroy frame

You can inspect the bytecode using:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

Understanding this helps explain recursion depth limits and function call overhead.

---

2. What is the difference between `__new__` and `__init__`?

Many developers know `__init__`, but advanced interviews expect knowledge of **object creation mechanics**.

Concept

Method	Responsibility
`__new__`	Creates the object instance
`__init__`	Initializes the created object

Example

```
class Demo:

    def __new__(cls):
        print("Creating instance")
        return super().__new__(cls)

    def __init__(self):
        print("Initializing instance")

obj = Demo()
```

Output

```
Creating instance
Initializing instance
```

When is `__new__` useful?

- Implementing **Singleton pattern**
- Subclassing **immutable types**
- Custom memory allocation

---

3. Explain Python's Global Interpreter Lock (GIL)

The GIL ensures that only one thread executes Python bytecode at a time within a process.

Why it exists

- Simplifies memory management
- Protects reference counting
- Prevents race conditions inside the interpreter

Example showing limitation

```
import threading

def task():
    for _ in range(10_000_000):
        pass

t1 = threading.Thread(target=task)
t2 = threading.Thread(target=task)

t1.start()
t2.start()

t1.join()
t2.join()
```

Even with two threads, CPU-heavy tasks do not run truly in parallel.

Real-world solutions

- `multiprocessing`
- `asyncio`
- Native extensions (C/C++)

---

4. What are Python descriptors?

Descriptors power many advanced Python features such as:

- properties
- methods
- class attributes
- ORM frameworks

A descriptor implements any of:

- `__get__`
- `__set__`
- `__delete__`

Example

```
class PositiveNumber:

    def __get__(self, instance, owner):
        return instance._value

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("Value must be positive")
        instance._value = value


class Account:
    balance = PositiveNumber()

acc = Account()
acc.balance = 100
```

This allows **fine-grained control over attribute access**.

---

5. How does Python manage memory?

Python uses two main mechanisms:

1. Reference Counting

Each object tracks how many references point to it.

```
import sys

a = []
print(sys.getrefcount(a))
```

2. Garbage Collector

Handles cyclic references that reference counting cannot clean.

```
import gc
gc.collect()
```

Important interview insight

Memory leaks can occur when:

- objects participate in reference cycles
- `__del__` methods are used improperly

---

6. What is the difference between generators and iterators?

Iterator

An object implementing:

- `__iter__`
- `__next__`

Generator

A simpler way to create iterators using yield.

Example

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for number in count_up_to(3):
    print(number)
```

Advantages of generators:

- Lazy evaluation
- Lower memory usage
- Ideal for large datasets and streaming

---

7. What are metaclasses in Python?

A metaclass defines how classes themselves are created.

If objects are instances of classes, then **classes are instances of metaclasses.**

Example

```
class Meta(type):

    def __new__(cls, name, bases, attrs):
        attrs['version'] = "1.0"
        return super().__new__(cls, name, bases, attrs)


class App(metaclass=Meta):
    pass


print(App.version)
```

Metaclasses are commonly used in:

- ORMs
- frameworks
- API validation systems

Final Thoughts

Modern Python interviews are no longer about remembering syntax. They focus on:

- **Understanding Python internals**
- **Performance trade-offs**
- **Writing scalable and maintainable systems**

If you can confidently explain topics like:

- descriptors
- metaclasses
- the GIL
- memory management
- generators

then you are already operating at a **senior Python engineering level**.

> Keep building. Keep experimenting. Python rewards those who explore its depths.